

Autotron Backend Pentest Assessment Guide

Architecture reconstruction, attack-surface mapping, workflow/state testing, API assessment strategy, and the knowledge required from the tester

Scope represented in this guide: NG / Oxygen entry flow, Autotron backend processing, NIMS estimate/rate integration, GeoHub, ORSM and API Gateway/mTLS dependencies, estimate creation, authorisation, Job Pack generation/review and release.

Important: This is an assessment strategy, not a statement that a vulnerability exists. Every hypothesis below must be verified against the actual HTTP requests, responses, roles, and business rules in the authorised test environment.

Prepared from the architecture and workflow information provided during the assessment discussion.

1. Executive Summary

Autotron should not be approached like a conventional web application consisting primarily of pages and browser forms. The useful security model is a backend workflow/API system in which NG appears to own much of the user-facing project/task workflow while Autotron performs automated processing at specific transitions, especially around estimate creation.

The central pentest question is therefore:

Who can trigger a workflow, what data can they control, what downstream systems does Autotron trust, what state transitions can they cause, and what privileged/business outcome is produced?

The most valuable areas are expected to be authorization, workflow/state integrity, financial/estimate integrity, replay and idempotency, cross-service trust, response validation, and the security of document generation. Generic vulnerabilities such as XSS or SQL injection still belong in the assessment, but they should not determine the initial strategy.

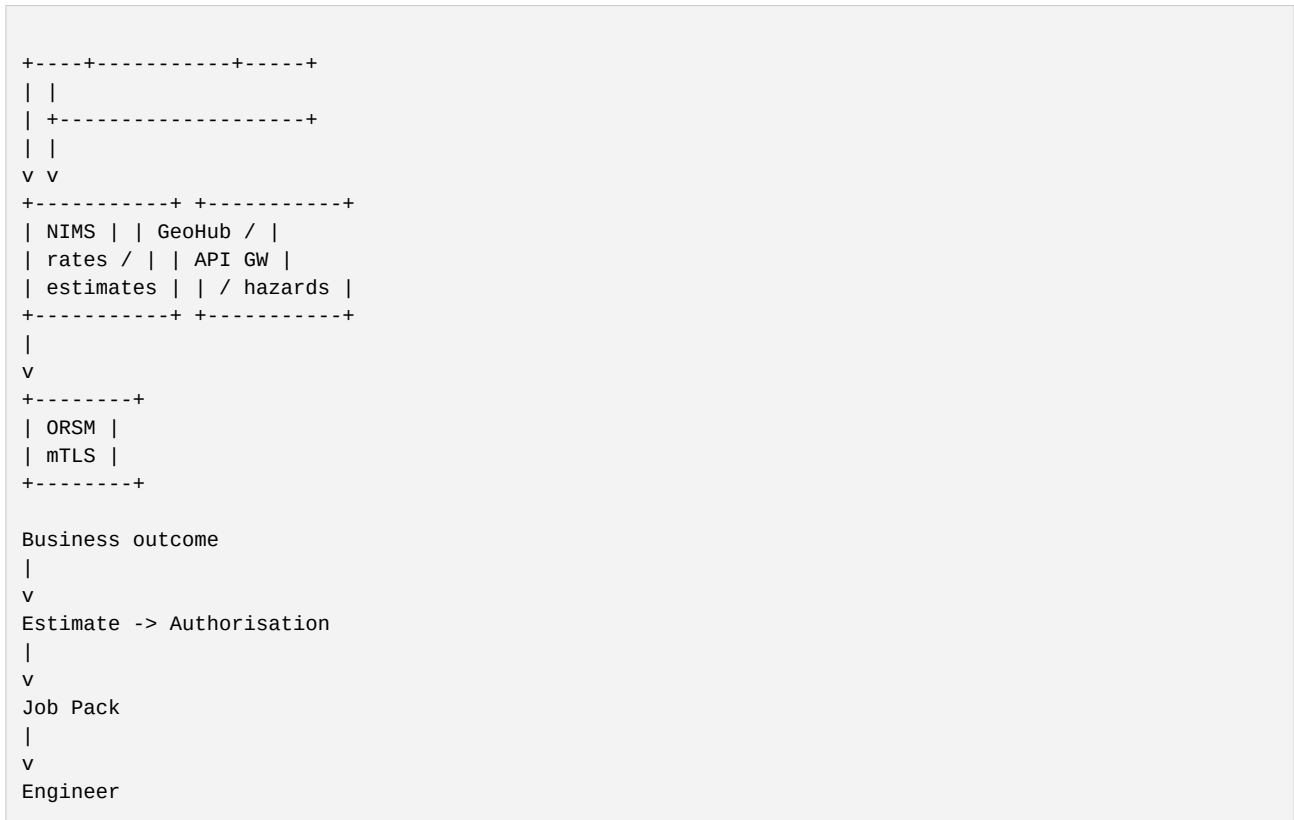
What the current evidence suggests

- Oxygen creates or contributes to project creation; NG then exposes the project/task workflow.
- A project is associated with a Project ID and is delivered into a DAN Planner queue.
- The NG UI exposes project/task information, NIMS-related information, Job Pack Review, additional estimates, estimate cancellation, folders and audit information.
- The user reaches a workflow where Create Estimate can be triggered.
- That action appears to cause an Autotron validation/processing call, after which estimate/task information is returned to or reflected in NG.
- Autotron communicates with downstream services including NIMS, GeoHub and ORSM. ORSM uses certificate-based two-way mTLS according to the supplied architecture notes; a GeoHub/API Gateway path is also described.
- NIMS appears to provide authoritative pricing/rate information that contributes to estimate creation and later authorisation.
- The final business outcome is not merely an API response: a validated/authorised estimate enables Job Pack creation/review and ultimately issue to an engineer.

2. Reconstructed Architecture

The following is a working model. It must be confirmed by capturing actual traffic because names and responsibilities inferred from the UI can differ from the implementation.

```
OXYGEN
|
| project creation / project information
v
+-----+
| NG |
| project/task workflow|
| queues / planner UI |
+-----+
|
| Create Estimate / workflow request
v
+-----+
| AUTOTRON |
| handler / validation |
| workflow processing |
| estimate automation |
| Job Pack generation |
```

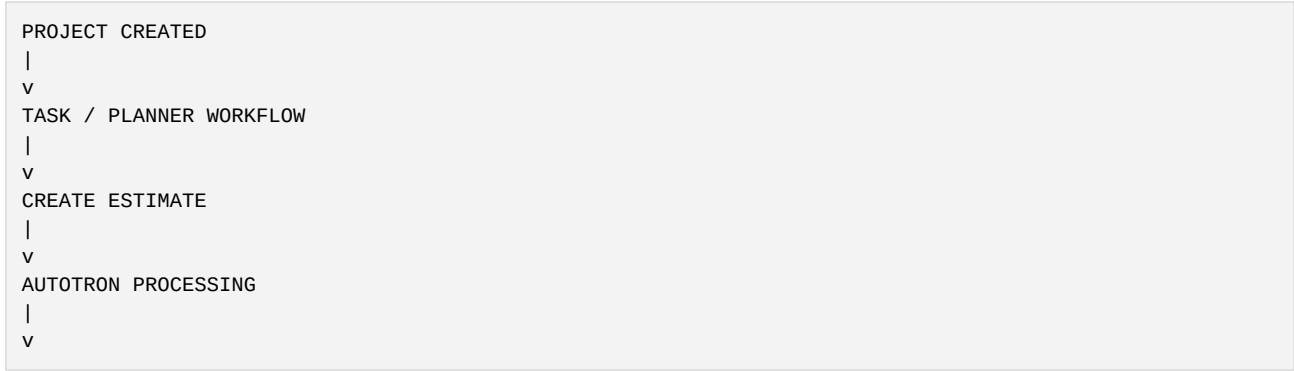


Trust boundaries

Boundary	What to establish	Why it matters
Oxygen → NG	Which fields/identities are created or inherited	Project ownership and identity may originate here.
NG → Autotron	Authentication, authorization, request schema, caller identity, project/task identifiers	Primary Autotron trust boundary.
Autotron → NIMS	How rates, estimates and identifiers are exchanged	Financial integrity and object authorization.
Autotron → GeoHub/API Gateway	Destination/resource control and authentication	Potential server-side request and trust issues.
Autotron → ORSM	mTLS identity, resource authorization and input validation	mTLS authenticates the connection but not every business action.
Autotron → document storage/folder	Where PDFs are stored and who can retrieve them	Sensitive data exposure and object authorization.

3. Model the Application as a State Machine

The screenshots and notes suggest a workflow in which project/task data moves through estimate creation, possible update, authorisation, Job Pack review and release. The exact states and transitions must be confirmed with the application owner.



```

ESTIMATE CREATED
|
v
ESTIMATE UPDATED (if applicable)
|
v
AUTHORISATION
|
+-----> FINANCIAL REJECTED
|
v
APPROVED / AUTHORISED
|
v
JOB PACK CREATED
|
v
JOB PACK REVIEW
|
v
RELEASE / ISSUE
|
v
FIELD ENGINEER

```

Why state is more important than individual endpoints

A request can be syntactically valid and still be a security defect if it moves an object into a state that the caller is not entitled to reach. For example, an authenticated planner might legitimately update an estimate but should not necessarily be able to authorise it, release it, or modify its financial values after authorisation.

For every state transition, determine:

- Who is permitted to invoke it?
- Which project/estimate/job objects may they operate on?
- What state must exist before the operation?
- What state is produced afterwards?
- Can the transition be invoked directly without completing previous steps?
- Can it be replayed?
- What happens if a downstream dependency fails during the transition?
- Is the server authoritative, or does it accept client-supplied status/approval fields?

4. First Practical Deliverable: API Inventory

Before aggressive testing, build an inventory from Burp HTTP history, Swagger/OpenAPI/WSDL material if available, application documentation, and conversations with the owners.

Field	Example / question
Endpoint	Actual path and host
Method	GET / POST / PUT / PATCH / DELETE
Caller	Browser/NG, service, background worker
Authentication	Token, session, mTLS, gateway, none
Authorization	Role/scope/project-level rule
Input	Path, query, headers, JSON/XML, file

Field	Example / question
Output	JSON/XML/PDF/status
State change	None / estimate / approval / Job Pack / release
Object identifiers	Project ID, task ID, estimate ID, document ID
Downstream dependency	NIMS / GeoHub / ORSM / other
Sensitive data	Rates, project data, engineer/customer data
Idempotency	Can the same request safely be repeated?

The most important transaction to capture first

Use a legitimate test project and intercept the exact transaction generated when the user clicks **Create Estimate**. Do not start by guessing Autotron endpoints.

```

Browser / NG
|
| click "Create Estimate"
v
HTTP request(s)
|
+--> NG endpoint?
|
+--> Autotron endpoint?
|
+--> token / session / headers?
|
+--> projectId / task / estimate fields?
|
+--> synchronous response or request/job ID?
|
v
HTTP response(s)
|
+--> estimate data?
+--> task/validation result?
+--> status transition?
+--> downstream identifiers?

```

Capture the request, response, timestamps, project ID, task ID, estimate ID, HTTP status and any correlation/request IDs. Redact credentials before sharing evidence.

5. Data Provenance: Track Where Every Important Field Comes From

Do not assume that a field displayed in NG is authoritative. Trace its origin through the request/response chain.

Data	Likely source from current understanding	Security question
Project ID	Oxygen / NG	Can another project ID be substituted?
ALECO/reference	Project workflow / CPR-related population	Is it merely a reference or does it control access/business logic?
Task type	NG / project task	Can a caller select an unauthorised task or change the type?
Location	NG/project data	Can it be changed to cause another resource lookup?
Hazard data	ORSM via service path	Does Autotron validate the response and requested location?

Data	Likely source from current understanding	Security question
Rates/cost	NIMS	Which system is authoritative? Can client values override NIMS?
Quantity/items	NG / Autotron / NIMS	Can totals be manipulated?
Approval status	Workflow/SOCS/NG	Can client-controlled fields mark something approved?
Job Pack	Autotron	Who can create/download/release it?

Core principle: for every financial or authorization-sensitive field, identify the authoritative source and verify that lower-trust callers cannot override it.

6. Authentication Testing

NG → Autotron

The supplied notes mention a token that is generated for approximately 30 minutes and then used with the handler API. The implementation needs to be confirmed.

- Determine who issues the token and who validates it.
- Determine whether the token represents a human user, NG service, or both.
- Check expiry handling around the 30-minute lifetime.
- Test missing, malformed, expired and otherwise invalid tokens in the authorised test environment.
- Determine whether tokens intended for another service/environment are accepted.
- If JWT is used, inspect claims such as issuer, audience, subject, expiry, scopes and roles; test whether server-side authorization actually enforces them.
- If mTLS is used for service-to-service communication, verify certificate identity, trust chain, expiry/revocation behavior and service-level authorization.

Important distinction

Authentication asks who is calling. Authorization asks what that caller may do. A valid NG/service token does not prove that every project, estimate or workflow action is permitted.

7. Authorization / BOLA / Object-Level Access

Given the repeated use of Project IDs and other business identifiers, object-level authorization should be one of the highest-priority test areas.

Horizontal access

Using two authorised test identities/roles where possible, create or obtain two distinct test projects and compare requests.

```
User A -> Project A -> Estimate A
User B -> Project B -> Estimate B

Controlled substitutions:
projectId A -> B
taskId A -> B
estimateId A -> B
jobId A -> B
documentId A -> B
```

- Read another project's data.
- Modify another project's data.

- Create an estimate for another project.
- Cancel or update another project's estimate.
- Download another project's Job Pack.
- Release or issue another project's Job Pack.

The most serious cases are not read-only IDORs but cross-project actions that change financial or operational state.

Vertical authorization

- Can a planner perform an authorisation action intended for a manager/approver?
- Can a user release/issue a Job Pack without the required approval?
- Can a normal workflow actor cancel an estimate or alter post-approval information?
- Can a lower-privilege user invoke Autotron directly even if NG hides the UI control?

8. Business-Logic and Workflow Testing

Test	Expected security property
Update before create	Rejected unless explicitly supported.
Authorise before valid estimate	Rejected.
Release before authorisation	Rejected.
Modify after authorisation	Rejected or forces re-authorisation according to business rules.
Cancel then authorise	Rejected.
Cancel then release	Rejected.
Create estimate twice	No unintended duplicate financial object.
Release twice	No duplicate operational issuance.
Use an old request after state changes	Server rechecks current state.
Change project ID mid-flow	Server verifies object ownership and relationship.

Workflow bypass hypothesis

One of the strongest hypotheses is not a classic injection attack but a state-transition bypass:

```
Expected:
Estimate created -> validated -> authorised -> Job Pack -> release

Potential defect:
Estimate created -> direct release
or
Financial rejected -> release
or
Approved -> client changes cost -> release
```

Whether any of these are valid depends on the actual business rules. The test is to establish those rules and verify the server enforces them rather than relying on the UI.

9. Estimate and Financial Integrity

This deserves a dedicated test stream because the application appears to calculate estimates and obtain rates/cost information from NIMS before authorisation.

Establish the authoritative source

Trace each value:

```
NG input
|
v
Autotron
|
+--> NIMS: item/rate/cost?
|
v
Autotron calculation?
|
v
Estimate
|
v
Authorisation
```

- Determine whether rate, item, quantity, currency and total are supplied by the client or derived from NIMS.
- Test whether client-supplied financial fields are ignored, validated or trusted.
- Check whether an estimate can be changed after authorisation.
- Check whether the project, estimate and NIMS identifiers remain bound to one another.
- Test negative, zero, unusually large and high-precision quantities/values in the test environment.
- Verify that currency and unit mismatches cannot produce an unintended total.
- Check whether a downstream failure causes a safe rejection rather than use of stale/default values.

Do not call a modified client-side number a vulnerability until you establish that the modified value actually influences an authoritative business outcome.

10. Replay, Idempotency and Race Conditions

Replay

Side-effecting operations should be tested for safe repetition.

- Create Estimate: send the same request twice.
- Update Estimate: repeat the same update.
- Authorise: repeat the operation.
- Job Pack generation: repeat the operation.
- Release/issue: repeat the operation.

Determine whether duplicates are intentionally prevented, safely deduplicated, or correctly treated as separate business transactions.

Race conditions

Focus concurrency on meaningful state transitions rather than indiscriminate load.

```
Request A: change estimate
Request B: authorise estimate

or

Request A: authorise
Request B: change cost

or

Request A: release
```


Request B: cancel

The critical question is whether the system can authorise or release a state different from the one that was actually reviewed.

11. Downstream Integration Security

Autotron → NIMS

- Verify service authentication and authorization.
- Determine whether identifiers are bound to the originating project.
- Verify response integrity and schema validation.
- Check whether malformed/partial/time-out responses cause fail-closed behavior.
- Check whether Autotron trusts returned status/cost/approval fields without validation.

Autotron → GeoHub / API Gateway

- Determine exactly which request components are attacker-influenced: URL, host, path, query, coordinates or resource ID.
- If a user can influence a destination or server-side resource request, assess for server-side request behavior/SSRF-like conditions within the authorised environment.
- Verify that the API Gateway authentication does not become the only authorization control.
- Check whether Autotron validates that the response corresponds to the requested project/location/resource.

Autotron → ORSM

Two-way mTLS is a strong transport/client-authentication control, but it does not automatically prove resource-level authorization.

- Verify certificate identity and trust configuration.
- Determine what request parameters Autotron controls.
- Determine whether arbitrary resources/locations can be queried.
- Verify response validation and safe handling of errors/timeouts.

12. Job Pack / PDF Generation

The generated Job Pack is an output, but its generation and retrieval are part of the attack surface.

- Identify the PDF/template generation technology.
- Identify every user-controlled field inserted into the document.
- Determine whether the document is generated from HTML, XML, templates or another representation.
- Assess the relevant injection class for that technology rather than blindly applying unrelated payloads.
- Test whether one project/user can download another project's Job Pack.
- Check for sensitive data exposure in generated documents.
- Check whether uploaded/replaced Job Pack or support documents have appropriate type, size, path and authorization controls.
- Verify that release/issue cannot be performed against an unapproved or wrong project's document.

13. Error Handling and Fail-Closed Behavior

Deliberately induce controlled failures in the test environment and observe the workflow.

Failure	Question
Invalid project ID	Does processing stop safely?
Invalid task	Does Autotron reject it?
NIMS unavailable	Does the estimate fail closed?
NIMS timeout	Can stale/default cost be used?
ORSM unavailable	Can Job Pack generation continue without required safety data?
Malformed downstream response	Is it rejected and logged safely?
Authorization failure	Does the workflow remain unapproved?
Repeated request	Does retry create unintended side effects?

Also inspect whether errors disclose internal URLs, service names, stack traces, tokens, certificate information, database details or other implementation secrets.

14. Input Validation and Injection

After understanding the data flow, test inputs according to their function.

Input category	Examples of testing focus
Object IDs	Other IDs, nonexistent IDs, boundary values, type confusion
Strings	Encoding, length, control characters, markup where relevant
JSON/XML	Parser edge cases, duplicate keys, unexpected structures
Numeric fields	Zero, negative, large, precision, type changes
Dates	Boundary dates, invalid ranges, timezone/format issues
File uploads	Type validation, size, filename/path handling, authorization
Headers	Unexpected trust headers, forwarded identity headers

Only escalate an injection hypothesis after identifying the component that interprets the input. For example, a string inserted into a PDF template has a different attack surface from a string used in a database query.

15. Recommended Assessment Phases

Phase	Objective	Output
1. Recon / architecture	Map components, endpoints, trust boundaries and workflow	Architecture + API map
2. Baseline capture	Capture legitimate Create Estimate and adjacent transactions	Golden request/response set
3. Authentication	Verify caller identity controls	Auth test results
4. Authorization	Test user/project/object/action permissions	BOLA/vertical auth matrix
5. Workflow	Test state transitions and bypass	State-transition matrix
6. Financial integrity	Trace rates/items/totals and authority	Financial trust map
7. Integration	Assess NIMS/GeoHub/ORSM trust boundaries	Service-to-service findings
8. Documents	Assess generation, storage, download/release	Document security results

Phase	Objective	Output
9. Input/injection	Test relevant parsers and interpreters	Injection findings
10. Replay/race/resilience	Verify side-effect safety and failure behavior	Concurrency/reliability findings
11. Retest	Verify fixes and workflow integrity	Retest evidence

16. Practical Test Matrix

ID	Area	Test hypothesis	Priority
AT-01	Auth	Unauthenticated/direct Autotron call is rejected	High
AT-02	Auth	Expired 30-minute token is rejected	High
AT-03	Authorization	Project ID cannot be swapped to another user's project	Critical/High
AT-04	Authorization	Estimate ID cannot be swapped across projects	High
AT-05	Authorization	Planner cannot invoke approval/release without privilege	Critical
AT-06	Workflow	Authorisation cannot occur from an invalid state	Critical
AT-07	Workflow	Release cannot occur before required approval	Critical
AT-08	Integrity	Client cannot override authoritative NIMS rate	Critical/High
AT-09	Replay	Duplicate Create Estimate does not create unintended duplicates	High
AT-10	Replay	Duplicate release does not cause duplicate issuance	Critical/High
AT-11	Race	Authorisation and update cannot produce an unreviewed approved state	Critical
AT-12	Integration	Malformed/failed NIMS response fails closed	High
AT-13	Integration	Downstream resource access is authorized	High
AT-14	Document	Job Pack download is object-authorized	High
AT-15	Document	Generated document does not expose unintended data	Medium/High
AT-16	Input	Relevant fields are protected against parser/injection issues	Risk-dependent

17. What You Actually Need to Know

You do **not** need to understand the entire Openreach business process before starting the technical assessment. You need enough business and technical knowledge to know what a request is allowed to do and what a successful response changes.

A. Must know before serious testing

Knowledge	Why you need it	How to obtain it
Actual Create Estimate request	Defines the primary Autotron entry point	Burp capture in test environment
Authentication mechanism	Determines how direct calls are protected	Token generation/API docs/traffic
Project ID semantics	Required for object-level authorization testing	Owner explanation + traffic
Task/estimate IDs	Needed for object and state testing	Request/response inspection
Valid workflow states	Needed to test state bypass correctly	Owner/state diagram
Roles	Needed for horizontal/vertical authorization	Role matrix from owner
NIMS authority	Determines whether rate/cost tampering matters	Architecture owner + captured calls

Knowledge	Why you need it	How to obtain it
Job Pack lifecycle	Determines impact of document/release actions	Workflow owner
Direct Autotron accessibility	Defines realistic attacker position	Network/API architecture

B. Useful technical knowledge

- HTTP request/response fundamentals: methods, headers, cookies, content types and status codes.
- Burp Suite: Proxy/HTTP history, Repeater, comparing requests and controlled parameter changes.
- REST APIs and JSON; XML if the application uses XML.
- Authentication vs authorization; JWT concepts if JWT is used.
- API object-level authorization / BOLA (IDOR).
- Business-logic and state-machine testing.
- Replay/idempotency and basic race-condition methodology.
- TLS and basic mTLS concepts.
- How server-side applications consume downstream APIs.
- Basic document/PDF generation concepts if Job Pack generation is in scope.
- How to distinguish an observed anomaly from an exploitable security impact.

C. You do NOT need to know immediately

- Every internal implementation detail of NIMS, ORSM or GeoHub.
- Every possible OWASP vulnerability category before starting.
- The meaning of every business abbreviation such as ALECO before you capture the data flow.
- The complete database schema.
- The complete source code.
- Every field visible in the NG UI.

The highest-value knowledge is the **contract**: who can call what, on which object, in which state, with which authoritative values, and with what business consequence.

18. Questions to Ask the Application Team

Category	Question
Architecture	What are the Autotron endpoints and hosts? Is a gateway in front of them?
Access	Can a tester call Autotron directly, or only through NG?
Auth	Who issues the 30-minute token and what identity does it represent?
AuthZ	Where is project-level authorization enforced: NG, gateway, Autotron, or multiple layers?
Workflow	What are the valid states and transitions for an estimate and Job Pack?
Roles	Which roles may create, update, approve, cancel, review and release?
IDs	Which identifiers are authoritative and how are they related?
ALECO	Is ALECO only a reference or does it affect access/business decisions?
NIMS	Which fields are returned by NIMS and which are authoritative?
NIMS	Can an estimate be changed after authorisation?

Category	Question
ORSM	What resources can Autotron request and how is access controlled?
GeoHub	Is the API Gateway the only security control or is resource authorization also performed?
Job Pack	Which technology generates PDFs and where are documents stored?
Files	What upload/download APIs exist and what authorization is expected?
Failure	What should happen if NIMS/ORSM/GeoHub is unavailable?
Audit	Which actions should appear in the Audit Trail and what identity is recorded?

19. Evidence to Capture During the Pentest

- Full request/response pair for every meaningful finding.
- Before/after state of the affected project/estimate.
- User/role used for the test.
- Project ID and object IDs used, using test data where possible.
- Correlation/request IDs and timestamps.
- Evidence showing the expected authorization/state rule.
- Evidence showing the unauthorized outcome or business impact.
- For financial issues, the authoritative rate/cost source and the final persisted value.
- For workflow issues, the state before the request and the state after it.
- For Job Pack issues, evidence of the document/resource accessed or altered and the authorization boundary.

Avoid weak evidence

A response code of 200 is not automatically a vulnerability. Likewise, a parameter changing in the response is not automatically a security issue. Tie each result to an authorization rule, integrity requirement, confidentiality boundary, or business-state violation.

20. A Practical First-Day Plan

Step	Action	Stop condition / output
1	Get two suitable test identities/roles if available.	Known role differences.
2	Create or obtain a clean test project.	Known Project ID and task.
3	Open Burp and capture normal NG traffic.	Baseline HTTP history.
4	Click Create Estimate.	Identify exact request(s).
5	Save request/response and correlation IDs.	Golden transaction.
6	Identify project/task/estimate identifiers.	Object map.
7	Trace whether Autotron calls NIMS/GeoHub/ORSM.	Dependency map.
8	Repeat with the same request once.	Replay/idempotency result.
9	With owner-approved test data, swap one object ID.	Initial BOLA result.
10	Attempt a state-invalid operation safely.	State enforcement result.
11	Compare client-supplied vs downstream-authoritative financial fields.	Financial trust map.
12	Document what remains unknown.	Questions for application owner.

21. The Mental Model to Use While Testing

For every API request, ask these seven questions:

1. WHO is calling?
2. WHAT object are they acting on?
3. WHAT state is that object currently in?
4. WHAT fields can the caller control?
5. WHICH system is authoritative for each important value?
6. WHAT downstream system will trust this request/response?
7. WHAT business outcome changes if the request succeeds?

If you can answer those seven questions for the Create Estimate transaction, you will have the foundation needed to expand the assessment to update, authorisation, cancellation, Job Pack review, download and release operations.

22. Final Assessment Checklist

Area	Complete?
Architecture and trust boundaries documented	■
NG → Autotron transaction captured	■
Authentication mechanism documented	■
Role/authorization matrix obtained	■
Project/object relationships mapped	■
Workflow state machine confirmed	■
Create/update/authorise/cancel/release APIs mapped	■
NIMS data provenance and authority confirmed	■
GeoHub/API Gateway flow confirmed	■
ORSM mTLS and resource authorization confirmed	■
Replay/idempotency tested	■
Race/state transition integrity tested	■
Job Pack generation/storage/download tested	■
Error/fail-closed behavior tested	■
Relevant injection/input cases tested	■
Audit trail behavior reviewed	■
Findings tied to concrete business impact	■

23. Bottom Line

The assessment should be organized around the **workflow and trust boundaries**, not around the fact that the application has a browser UI.

Your most important first move is to capture the exact request generated when the NG user presses **Create Estimate**. Once that request is known, you can identify the Autotron handler, its authentication, its parameters, its object identifiers, the downstream NIMS/GeoHub/ORSM interactions, and the state transition it causes.

From there, the assessment naturally expands outward:

```
Create Estimate
|
```

```
+--> authentication
+--> object authorization
+--> parameter integrity
+--> workflow/state enforcement
+--> replay/idempotency
+--> downstream trust
+--> financial integrity
+--> document generation
+--> error handling
|
v
business outcome
```

That is the core strategy: **understand the legitimate transaction first, establish what the server should permit, then deliberately test whether the implementation permits more than it should.**